

Desarrollo de una Arquitectura Modular para Modelos Digitales Fenomenológicos en Python: Validación, Pruebas y Replicabilidad

Autor(es)
Departamento de Ingeniería

Resumen

Este artículo presenta la arquitectura computacional de un Modelo Digital Fenomenológico desarrollado en Python, cuyo diseño prioriza la modularidad, la validación en múltiples capas y la replicabilidad en diferentes industrias de proceso. El sistema se organiza en tres módulos independientes y acoplados: un núcleo de balances fenomenológicos, un motor de restricciones de capacidad y una interfaz de visualización en Streamlit. Las constantes de equipo, los límites de control y los factores de seguridad están externalizados en archivos de configuración con validación automática de rangos físicos. Un conjunto de once pruebas unitarias verifica la integridad del sistema ante modificaciones. Como caso de validación, se instanció la arquitectura para una planta de Proteína Aislada de Soya (ISP) de 1000 kg/h. Una campaña de Monte Carlo sobre el diseño base inicial arrojó solo 4,70 % de éxito operativo; tras redimensionar los equipos críticos, una segunda campaña de 300,000 corridas alcanzó 99,88 % de éxito ($3,03\sigma$), validando la utilidad del enfoque para diagnóstico y optimización de procesos.

Palabras Clave

Modelo Digital, Arquitectura de Software, Python, Replicabilidad, Monte Carlo, ISO 23247.

1. Introducción

La taxonomía de Kritzinger et al. [2] establece tres niveles de integración virtual: **Modelo Digital** (representación estática sin intercambio automático de datos), **Sombra Digital** (flujo unidireccional de datos del proceso al modelo) y **Gemelo Digital** (flujo bidireccional con control predictivo en tiempo real). La mayoría de los desarrollos reportados en la literatura corresponden a los dos niveles superiores, pero dependen de infraestructuras de datos comerciales que dificultan su replicabilidad independiente [3]. Existe una brecha en la disponibilidad de plataformas de código abierto para la capa base de esta jerarquía: el Modelo Digital fenomenológico.

Este trabajo aborda esa brecha presentando la arquitectura computacional de un Modelo Digital Fenomenológico modular y replicable. El sistema se posiciona explícitamente en el primer nivel de la taxonomía de Kritzinger: no requiere flujo de datos en tiempo real ni infraestructura IoT, sino que constituye la representación matemática validable que sirve como cimiento para una futura transición hacia una Sombra o Gemelo Digital. Su núcleo de cálculo se basa en ecuaciones de conservación de masa y energía, no en correlaciones empíricas o modelos de caja negra, lo que lo hace interpretable, depurable y transferible entre procesos [22].

El artículo se estructura como sigue. La sección 2 revisa el estado del arte en arquitecturas modulares y presenta una comparativa con frameworks existentes, clasificándolos según su nivel de integración. La sección 3 detalla la arquitectura propuesta y su formalización matemática. La sección 4 describe los requisitos de replicabilidad. La sección 5 presenta la verificación numérica mediante el caso de una planta ISP. La sección 6 concluye y discute el trabajo futuro.

2. Trabajo Relacionado y Comparativa

El desarrollo de frameworks open-source para gemelos digitales ha crecido significativamente en los últimos años [3]. A continuación se revisan los trabajos más cercanos al enfoque propuesto.

2.1 Frameworks de Código Abierto

OpenTwins [4] es un framework open-source para el desarrollo de DT composicionales, basado en Eclipse Ditto como núcleo e integrando IoT, ML y visualización 3D. Publicado en *Computers in Industry* (2023), presenta una arquitectura modular que permite componer DT de subsistemas individuales en un DT de orden superior. Su caso de validación en la industria petroquímica demuestra su aplicabilidad en procesos industriales reales. A diferencia de nuestro enfoque, OpenTwins requiere una infraestructura de contenedores (Kubernetes) y no incorpora un modelo fenomenológico de balances de materia y energía.

CoFMPy [5] es un framework Python para prototipado rápido de DT basados en co-simulación FMI (Functional Mock-up Interface). Desarrollado por IRT Saint-Exupéry, permite integrar FMUs con componentes de IA para crear DT híbridos. Su caso de estudio en monitoreo de baterías para aeronaves híbridas demuestra la viabilidad del enfoque. CoFMPy es la alternativa tecnológicamente más cercana a nuestro proyecto por su implementación en Python puro.

DT-Forge [6] propone una arquitectura de seis capas (datos, simulación, reactiva, inteligente, autónoma) con subsistemas transversales de conectores y agregación. Es domain-agnostic y está diseñado para gemelos que ingieren telemetría en tiempo real. Su filosofía arquitectónica es similar a la nuestra, aunque su complejidad operativa es significativamente mayor al requerir Docker, MQTT, InfluxDB, MongoDB, Neo4j y otros servicios.

UniTwin [7] presenta un enfoque de DT universales containerizados en entornos Kubernetes, con énfasis en reconfigurabilidad dinámica y escalabilidad. Un estudio comparativo reciente [8] lo evalúa frente a Azure Digital Twins, Eclipse Ditto y White Label DT, destacando su modularidad y eficiencia en tiempo real.

TumorTwin [9] es un framework Python para DT personalizados en oncología, con una arquitectura modular que permite componer diferentes modelos de crecimiento tumoral, solvers y algoritmos de optimización. Aunque su dominio de aplicación es radicalmente distinto, comparte con nuestro proyecto el uso de Python como lenguaje único y el énfasis en la modularidad para facilitar la experimentación.

2.2 DT en Procesos de Biorefinería y Alimentos

En el dominio de procesos, **Udugama et al.** [10] proponen un modelo digital multi-capa para la industria alimentaria que captura comportamiento desde la escala sensorial hasta la planta completa, con casos conceptuales en fermentación de queso crema y congelación de carne. **Gamero et al.** [11] presentan el concepto de *Lean Digital Twin* para biorefinerías, abordando los desafíos prácticos de adquisición y gestión de datos. **Verboven et al.** [12] sientan las bases conceptuales para la implementación de DT en operaciones de procesamiento de alimentos, identificando la necesidad de modelos de alta fidelidad con capacidad de simulación en tiempo real.

2.3 Comparativa

La Tabla 1 presenta una comparación de la arquitectura propuesta con los frameworks analizados, clasificándolos según su nivel de integración virtual [2].

Como se observa, la mayoría de los frameworks existentes carecen de validación estadística y pruebas unitarias automatizadas, dos elementos esenciales para garantizar la robustez y replicabilidad de un modelo digital en entornos de ingeniería de procesos. Nuestra propuesta es el único framework clasificado como Modelo Digital que incorpora simultáneamente las cuatro capas de validación. Esta comparativa no pretende establecer superioridad, sino identificar oportunidades de integración futura: un sistema híbrido podría, por ejemplo, combinar el núcleo fenomenológico de esta arquitectura con las capacidades de ingestión de datos en tiempo real de DT-Forge (transición a Sombra Digital) o la composición de gemelos de OpenTwins (transición a Gemelo Digital).

3. Arquitectura del Modelo Digital

El sistema se organiza en cuatro módulos computacionales en el paquete `core/` y un módulo de interfaz, todos desarrollados en Python 3.10+. La comunicación entre módulos se realiza exclusivamente mediante diccionarios con estructura definida, lo que permite reemplazar cualquier módulo sin afectar los demás.

3.1 Vista General de Módulos

- `core/equipment_specs.py` – Configuración de equipo. Alberga los diccionarios `EQUIPMENT_SPEC_DEFAULTS` (capacidades nominales de diseño), `EQUIPMENT_SPEC_LIMITS` (rangos físicos permisibles de cada parámetro),

`CAPACITY_LIMIT_DEFAULTS` y `CAPACITY_LIMIT_BOUNDS` (factores de seguridad operativos), junto con las funciones `validate_equipment_specs` y `validate_capacity_limits` que verifican automáticamente la integridad de todos los valores. Cualquier adaptación a un nuevo proceso comienza por modificar las claves y valores de este módulo.

- `core/stage_equations.py` – Núcleo fenomenológico. Contiene las funciones independientes `calc_stage_N` que resuelven los balances de materia y energía de cada etapa; los diccionarios `CONTROL_LIMITS` (rangos operativos de las variables de consigna) y `KPI_SPECS` (rangos objetivo de los indicadores de calidad por etapa); y la función `run_process_model` que orquesta la ejecución secuencial de todas las etapas, valida la integridad numérica (finitud, no negatividad), calcula el cierre del balance de masa global e invoca el motor de restricciones.
- `core/equipment_constraints.py` – Motor de restricciones. La función `evaluate_capacity_constraints` compara la carga solicitada por cada etapa contra la capacidad física del equipo correspondiente, ajustada por factores de seguridad, y genera alarmas estructuradas con código, mensaje y recomendación cuando se supera un umbral. La función `enforce_capacity_constraints` eleva una excepción `EquipmentCapacityError` ante cualquier violación.
- `core/process_model.py` – Soporte de historial. Proporciona la clase `ControlLog` (buffer circular en memoria para 1000 puntos temporales) y la función `build_snapshot` que aplanas controles, especificaciones y resultados por etapa en un diccionario plano para persistencia y visualización de series temporales.
- `app.py` – Interfaz de sala de control en Streamlit. Integra el bucle de simulación con paso temporal configurable, la inercia de primer orden sobre cada variable de proceso y la visualización en tiempo real de KPIs, alarmas, diagrama de flujo y selector de configuración de equipo.
- `tests/` – Once pruebas unitarias distribuidas en tres archivos que verifican la consistencia del balance de materia, los KPIs del caso base, la detección de cada restricción de capacidad y la integridad del fluido grama visual.

3.2 Configuración Externalizada y Validación

Todos los parámetros del proceso están externalizados en constantes de módulo con validación automática. El archivo `equipment_specs.py` define tres capas de configuración:

1. **Valores nominales** (`EQUIPMENT_SPEC_DEFAULTS`): Capacidades de diseño recomendadas para operación normal.
2. **Límites físicos** (`EQUIPMENT_SPEC_LIMITS`): Rangos [mín, máx] que garantizan que los parámetros sean físicamente realizables. La función `validate_equipment_specs` verifica que cada valor esté dentro de su rango.
3. **Límites operativos** (`CAPACITY_LIMIT_DEFAULTS`): Factores de seguridad (fracciones máximas de llenado, carga

Cuadro 1: Comparativa de Frameworks para Modelos Digitales

Framework	Python puro	Valid. Estad.	Pruebas Unit.	Modelo Fenom.	Nivel Integr.	UI	Caso Real
Este proyecto	✓	✓	✓	✓	Modelo	✓	✓
OpenTwins [4]	×	×	×	×	Gemelo	✓	✓
CoFMPy [5]	✓	×	×	FMU ext.	Sombra	GUI	✓
DT-Forge [6]	✓	×	×	Opcional	Gemelo	×	×
UniTwin [7]	×	×	×	×	Gemelo	✓	×
TumorTwin [9]	✓	×	×	✓	Modelo	×	✓

térmica, etc.) que definen el margen operativo antes de disparar una alarma.

Análogamente, cada variable de control u_j tiene asociado un rango operativo $[u_{j,\text{mín}}, u_{j,\text{máx}}]$. La validación previa a la ejecución del modelo se expresa como el conjunto de condiciones:

$$u_j \in \mathbb{R}, \quad |u_j| < \infty, \quad u_{j,\text{mín}} \leq u_j \leq u_{j,\text{máx}} \quad \forall j \quad (1)$$

Este patrón de validación en tres niveles (configuración, entrada, ejecución) garantiza que el modelo nunca opera con parámetros fuera de especificación.

3.3 Núcleo Fenomenológico

Cada etapa del proceso modela balances de materia y energía mediante ecuaciones algebraicas. Las ecuaciones presentadas a continuación representan los balances ideales de cada operación unitaria; la implementación computacional incorpora funciones de penalización por desviación de condiciones nominales (pH, temperatura, tiempo de residencia), un factor de pérdida operativa ($F_{\text{LOSS}}=2\%$ por etapa) y mapas de eficiencia empíricos que modifican los términos nominales en función del estado de las variables de control. La Figura 1 muestra las seis etapas principales y las subetapas de separación mecánica, ósmosis inversa y centrifugación que completan la secuencia del proceso ISP.

3.3.1 Etapa 0: Molienda y Mezcla

El slurry de alimentación se obtiene combinando soja y agua:

$$\dot{m}_0 = \dot{m}_{\text{soya}} + \rho_w Q_w \quad (2)$$

donde $\dot{m}_{\text{soya}}$ es el flujo másico de soja (1000 kg/h), $\rho_w \approx 1000 \text{ kg/m}^3$ la densidad del agua, y Q_w el caudal volumétrico de agua de mezcla.

3.3.2 Etapa 1: Lixiviación Alcalina

La proteína soluble se extrae en medio alcalino (pH 8.5). El residuo sólido (okara) retiene una fracción de humedad H_{okara} , y el extracto líquido global alimenta las etapas siguientes:

$$\dot{m}_{\text{prot}} = \eta_{\text{ext}} X_{\text{prot}} \dot{m}_{\text{soya}} \quad (3)$$

$$\dot{m}_{\text{okara}} = \frac{\dot{m}_{\text{soya}} (1 - \eta_{\text{ext}} X_{\text{prot}})}{1 - H_{\text{okara}}} \quad (4)$$

$$\dot{m}_{\text{ext}} = \dot{m}_0 - \dot{m}_{\text{okara}} \quad (5)$$

donde $\eta_{\text{ext}} = 0,88$ es la eficiencia de extracción nominal, $X_{\text{prot}} = 0,375$ la fracción másica de proteína en la soja, y

$H_{\text{okara}} = 0,65$ la humedad del okara. En la implementación, η_{ext} es modulada por funciones continuas que penalizan desviaciones de pH, temperatura, tiempo de residencia y relación sólido-líquido.

3.3.3 Etapa 2: Pasteurización HTST

El extracto líquido $\dot{m}_{\text{ext}}$ se calienta desde la temperatura de operación hasta la temperatura de pasteurización. El área de intercambio se calcula del balance energético:

$$\dot{Q}_{\text{HX}} = \dot{m}_{\text{ext}} C_{p,\text{ext}} (T_{\text{out}} - T_{\text{in}}) \quad (6)$$

$$A_{\text{HX}} = \frac{\dot{Q}_{\text{HX}}}{U \Delta T_{\text{lm}}} \quad (7)$$

con $T_{\text{in}} = 25^\circ\text{C}$, $T_{\text{out}} = 80^\circ\text{C}$, $C_{p,\text{ext}} = 3,9 \text{ kJ/kg}^\circ\text{C}$ la capacidad calorífica del extracto acuoso (correlación de Siebel para 7,6% de sólidos), U el coeficiente global de transferencia, y ΔT_{lm} la diferencia de temperatura media logarítmica.

3.3.4 Etapa 3: Concentración (OI + Evaporación)

La preconcentración por ósmosis inversa recibe el extracto pasteurizado $\dot{m}_{\text{ext}}$ y sigue el modelo de Kedem–Katchalsky [20]:

$$\dot{m}_{\text{perm}} = A_m A_w (\Delta P - \Delta \pi) \quad (8)$$

donde A_m es el área de membrana, A_w la permeabilidad hidráulica, ΔP la presión aplicada y $\Delta \pi$ la presión osmótica. El concentrado que alimenta al evaporador es $\dot{m}_{\text{conc}} = \dot{m}_{\text{ext}} - \dot{m}_{\text{perm}}$.

El evaporador al vacío remueve el agua remanente del concentrado:

$$\dot{Q}_{\text{ev}} = \lambda \dot{m}_{\text{vap}} = U_{\text{ev}} A_{\text{ev}} \Delta T_{\text{ev}} \quad (9)$$

con λ el calor latente de vaporización y $\dot{m}_{\text{vap}}$ el flujo de vapor generado. El concentrado final que avanza a precipitación es $\dot{m}_{\text{conc,out}} = \dot{m}_{\text{conc}} - \dot{m}_{\text{vap}}$.

3.3.5 Etapa 4: Precipitación Isoeléctrica y Centrifugación

La proteína remanente en el concentrado $\dot{m}_{\text{conc,out}}$ se precipita en el punto isoeléctrico (pH 4.5) ajustando el pH con ácido clorhídrico ($\dot{m}_{\text{acid}}$). La pasta húmeda se separa por centrifugación, dejando un suero residual:

$$\dot{m}_{\text{ppt}} = \eta_{\text{ppt}} \dot{m}_{\text{prot},4} \quad (10)$$

$$\dot{m}_{\text{ppt,húmedo}} = \frac{\dot{m}_{\text{ppt}}}{1 - H_{\text{pasta}}} \quad (11)$$

$$\dot{m}_{\text{suero}} = \dot{m}_{\text{conc,out}} + \dot{m}_{\text{acid}} + \dot{m}_{\text{wash}} - \dot{m}_{\text{ppt,húmedo}} \quad (12)$$

con $\eta_{\text{ppt}} = 0,95$ la eficiencia de precipitación nominal, $H_{\text{pasta}} = 0,50$ la humedad de la pasta centrifugada, y $\dot{m}_{\text{prot},4}$ la proteína disponible tras las etapas previas.

3.3.6 Etapa 5: Secado por Atomización

El balance de masa y energía en el secador determina el polvo final:

$$\dot{m}_{\text{powder}} = \dot{m}_{\text{ppt}} \frac{1 - X_{\text{moist},\text{in}}}{1 - X_{\text{moist},\text{out}}} \quad (13)$$

$$\dot{Q}_{\text{dry}} = \dot{m}_{\text{air}} C_{p,\text{air}} (T_{\text{in}} - T_{\text{out}}) + \lambda \dot{m}_{\text{evap}} \quad (14)$$

donde $X_{\text{moist},\text{in}} = 0,80$, $X_{\text{moist},\text{out}} = 0,05$, y $T_{\text{in}} = 185^\circ\text{C}$.

3.3.7 Inercia Dinámica

La respuesta temporal del modelo ante cambios en las variables de control sigue una ecuación diferencial de primer orden con ganancia unitaria:

$$\tau \frac{dy(t)}{dt} + y(t) = u(t) \quad (15)$$

Cuya solución analítica para entrada constante $u[k]$ en un intervalo Δt es el filtro exponencial:

$$y[k+1] = y[k] + (u[k] - y[k]) (1 - e^{-\Delta t/\tau}) \quad (16)$$

donde τ es la constante de tiempo del sistema, $u[k]$ el setpoint en el paso k , y $y[k]$ la variable de proceso medida. Cada variable de control tiene su propia constante τ_j , permitiendo capturar dinámicas heterogéneas en las distintas etapas.

Cada función de etapa ((2)–(14)) es independiente y puede ser modificada, reemplazada o validada de forma aislada. La ecuación (16) se aplica como paso posterior al cálculo del estado estacionario.

3.4 Motor de Restricciones

Las restricciones de capacidad se expresan como un sistema de inecuaciones que comparan la carga solicitada por el proceso contra la capacidad disponible del equipo, ajustada por factores de seguridad f_s :

$$V_{\text{TK}} \leq V_{\text{TK},\text{max}} \cdot f_{s,\text{TK}} \quad (\text{Tanques}) \quad (17)$$

$$\dot{Q}_{\text{HX}} \leq A_{\text{HX},\text{max}} \cdot U \cdot \Delta T_{\text{lm}} \cdot f_{s,\text{HX}} \quad (\text{Intercambiador}) \quad (18)$$

$$\dot{m}_{\text{vap}} \leq \dot{m}_{\text{vap},\text{max}} \cdot f_{s,\text{EV}} \quad (\text{Evaporador}) \quad (19)$$

$$\dot{V}_{\text{CF}} \leq \dot{V}_{\text{CF},\text{max}} \cdot f_{s,\text{CF}} \quad (\text{Centrifuga}) \quad (20)$$

$$\dot{m}_{\text{evap},\text{SD}} \leq \dot{m}_{\text{evap},\text{SD},\text{max}} \cdot f_{s,\text{SD}} \quad (\text{Secador}) \quad (21)$$

La violación de cualquiera de estas inecuaciones genera una alarma con el equipo y la magnitud del exceso. Este diseño desacopla la detección de fallas de la lógica del proceso: el modelo fenomenológico no necesita conocer las restricciones, y el motor de restricciones no necesita conocer los detalles del balance de materia.

3.5 Disponibilidad del Código Fuente

El código fuente completo de la arquitectura, incluyendo los módulos de balances fenomenológicos, motor de restricciones y batería de pruebas unitarias, está publicado en un repositorio público bajo licencia MIT. El repositorio incluye instrucciones detalladas para replicar el entorno computacional y ejecutar las simulaciones presentadas en este artículo. Puede accederse mediante el repositorio institucional del proyecto. Los experimentos de Monte Carlo son reproducibles ejecutando `python scripts/plot_monte_carlo.py`.

3.6 Interfaz Visual (app.py) y Bucle de Simulación

La interfaz en Streamlit implementa un bucle temporal que avanza en pasos discretos ($\Delta t = 1$ s). En cada paso:

1. Lee los setpoints ingresados por el operador.
2. Si hay cambios, ejecuta `run_process_model` para calcular el nuevo estado estacionario.
3. Aplica la inercia de primer orden a cada variable para obtener el valor transitorio.
4. Actualiza los indicadores y alarmas en la interfaz.
5. Espera el próximo paso (simulado o en tiempo real).

La visualización incluye: diagrama de flujo interactivo con balances de masa, KPIs en tarjetas numeradas, historial de variables versus setpoint y panel de alarmas FMEA. La estructura de datos del módulo de configuración (EQUIPMENT_SPEC_DEFAULTS) sigue el principio de submodelos de la Asset Administration Shell (IEC 63278) [18], y la interfaz de variables de control está diseñada para un acoplamiento futuro con OPC UA [17] dentro del marco de la norma ISO 23247 [14] para la eventual transición a una Sombra Digital. En la sesión actual, la aplicación en Streamlit ejecuta exclusivamente operaciones en memoria sobre el modelo fenomenológico, sin comunicación con sistemas externos ni servidores OPC UA.

3.7 Sistema de Pruebas Unitarias

El directorio `tests/` contiene once pruebas que verifican:

1. **Consistencia del balance de materia** (`test_balance_is_consistent_with_model_integrity`): El payload del fluido grama visual reproduce exactamente los totales de masa y el cierre del balance calculados por el núcleo fenomenológico.
2. **Rendimientos y flujos no negativos** (`test_all_rendimientos_are_non_negative`, `test_outputs_are_finite`): Todos los rendimientos del fluido grama y todos los valores numéricos del modelo son ≥ 0 y finitos.
3. **Caso base contra referencia** (`test_baseline_matches_project_final`): Con los parámetros nominales, los KPIs coinciden con los valores de referencia (ej. eficiencia de extracción 88%, polvo final 301,6 kg/h, rendimiento global 76,40%, error del cierre de masa $\leq 0,1\%$).

4. **Detección de bloqueos** (`test_dryer_capacity_blocks`, `test_interchanger_capacity_too_small_blocks`, `test_baseline_capacity_passes`): Especificaciones pequeñas disparan el código de error correcto; las nominales no disparan falsos positivos.

5. **Validación de controles** (`test_invalid_control_range_raises`, `test_non_finite_control_raises`): Valores fuera de rango o no finitos son rechazados con la excepción correspondiente.

Estas pruebas se ejecutan con `python -m unittest discover tests` y deben pasar antes de cualquier modificación al código, garantizando que no se introduzcan regresiones.

4. Requisitos de Replicabilidad

Para adaptar la arquitectura a una nueva industria de proceso, se deben seguir tres pasos, cada uno con un entregable de código específico:

4.1 Paso 1: Mapeo de Equipos y Configuración

Identificar los equipos del proceso y sus capacidades nominales. Crear un diccionario `EQUIPMENT_SPEC_DEFAULTS` con las capacidades de diseño y `CAPACITY_LIMIT_DEFAULTS` con los factores de seguridad operativos. La función `validate_equipment_specs` verificará que todos los valores estén dentro de rangos físicamente posibles. Este archivo es el único punto de contacto entre el código y la planta física.

4.2 Paso 2: Implementación de Balances

Para cada operación unitaria, implementar una función `stage_n(controls, specs) -> dict` que resuelva los balances de materia y energía. Si no se requiere un modelo fenomenológico detallado, puede implementarse una versión simplificada con balances globales o factores de rendimiento empíricos.

4.3 Paso 3: Definición de Variables de Control

Definir `CONTROL_LIMITS` con los rangos operativos de cada variable de control. Estas variables son los puntos de consigna que el operador puede modificar desde la interfaz. La función `validate_controls` garantizará que ningún valor fuera de especificación alcance el modelo.

La arquitectura está diseñada para que estos tres pasos sean independientes: se puede cambiar la configuración de equipos sin modificar los balances, y viceversa.

5. Verificación Numérica: Caso de Estudio ISP

Para verificar la arquitectura, se instanció el modelo digital para una planta de Proteína Aislada de Soya (ISP) con capacidad nominal de 1000 kg/h. El proceso comprende seis etapas principales, como se muestra en la Figura 1: molienda y mezcla de agua (Etap 0), lixiviación alcalina (Etap 1), pasteurización HTST (Etap 2), concentración por ósmosis inversa y evaporación al vacío (Etap 3), precipitación isoelectrica y centrifugación (Etap 4), y secado por atomización (Etap 5).

5.1 Verificación del Balance de Materia

Con los parámetros nominales ($F_{soya} = 1000 \text{ kg/h}$, $Q_{agua} = 12 \text{ m}^3/\text{h}$, pH 8.5, temperatura secado 185°C), el modelo produce los siguientes KPIs, consistentes con las pruebas unitarias:

- Eficiencia de extracción: 88,0 %
- Proteína precipitada: 292,3 kg/h
- Polvo proteico final: 301,6 kg/h (286,5 kg/h de proteína)
- Rendimiento global de proteína: 76,40 %
- Humedad residual del polvo: 5,0 %

El cierre del balance de materia global tiene un error inferior al 0,5 %, validando la consistencia interna del modelo.

5.2 Diagnóstico Estocástico del Diseño Base

Se ejecutaron 10 batches independientes de Monte Carlo (100,000 corridas) sobre el diseño base para evaluar su robustez. Las 19 variables de control se perturbaron aleatoriamente dentro de rangos amplios. Solo el 4,70 % de las corridas fueron exitosas ($\sigma = \pm 0,18 \%$, IC 95 %: [4,59 %, 4,81 %]). Los cuellos de botella identificados fueron el tanque de lixiviación TK-101 (78,23 % de las corridas con falla), el tanque de agua TK-100 (69,50 %), el evaporador EV-301 (57,06 %) y el pasteurizador HX-201 (40,69 %).

5.3 Redimensionamiento y Validación Final

A partir del diagnóstico, se redimensionaron los equipos críticos en la configuración (Tabla 2). Tras la reconfiguración, una campaña de 30 batches (300,000 corridas) con rangos operativos ampliados ($\pm 20 \%$ en alimentación, $\pm 25 \%$ en flujo de agua) arrojó los siguientes resultados:

- **Tasa de éxito:** 99,88 % (299,633 de 300,000 corridas exitosas).
- **Desviación estándar entre lotes:** $\pm 0,026 \%$, con IC 95 %: [99,87 %, 99,89 %].
- **Nivel Sigma:** $Z = 3,03\sigma$ (equivalente a $\sim 1200 \text{ DPMO}$), calculado mediante la función de distribución normal estándar inversa $\Phi^{-1}(1 - p_{\text{falla}})$ sobre la tasa de fallas residual $p_{\text{falla}} = 0,12 \%$.
- **Fallas residuales:** Solo `STAGE1_TANK_CAPACITY` (TK-101) en 0,12 %, bajo condiciones extremas de residencia > 100 minutos. Las cinco restricciones restantes no se activaron en ninguna corrida.

La Figura 2 muestra la distribución de las tasas de éxito por lote. La estrecha agrupación en torno al 99,88 % y la baja desviación estándar (0,026 %) confirman la reproducibilidad del resultado y la robustez del diseño optimizado.

El nivel $3,03\sigma$ ($\sim 1200 \text{ DPMO}$, calculado directamente sobre la tasa de fallas observada) refleja la alta confiabilidad del diseño optimizado dentro del dominio puramente estocástico simulado, y demuestra que la arquitectura permite diagnosticar, redimensionar y validar la robustez de un proceso de forma sistemática.

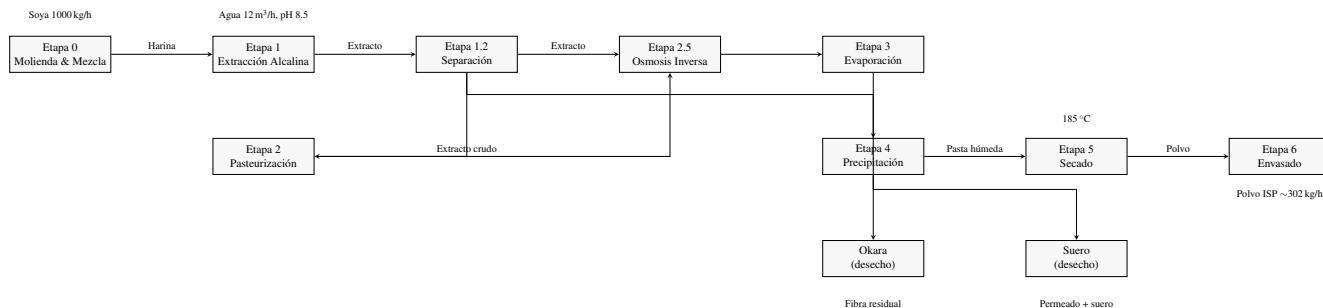


Figura 1: Diagrama de flujo del proceso ISP. Se muestran las seis etapas principales, las corrientes de entrada (soya, agua) y los subproductos (okara, suero).

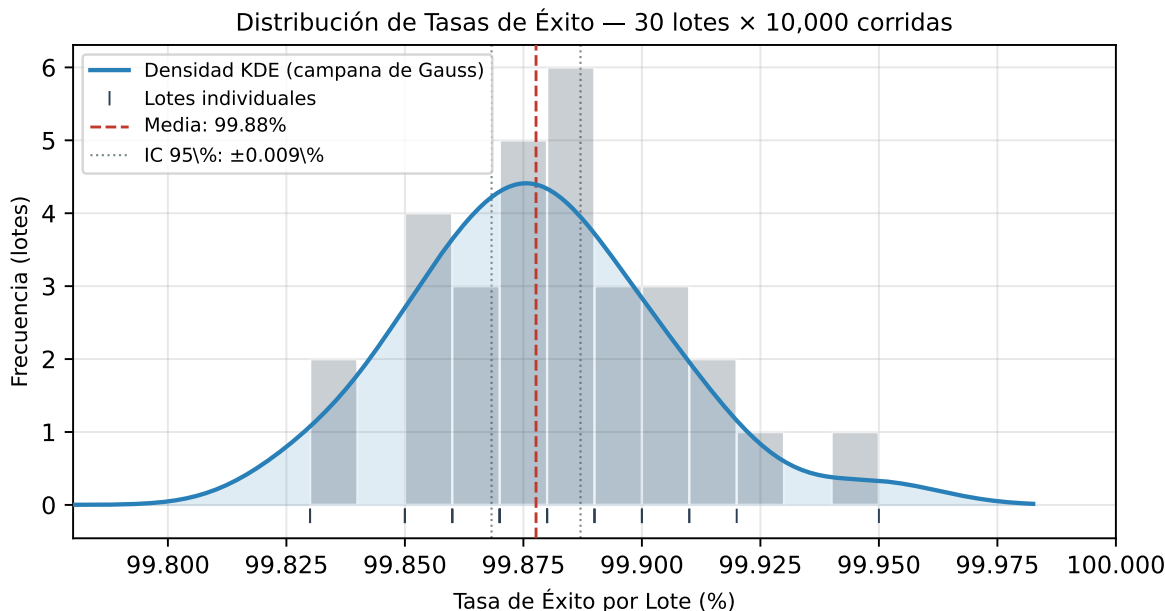


Figura 2: Distribución de las tasas de éxito por lote en la campaña optimizada (30 batches de 10,000 corridas cada uno). La curva suave representa la estimación de densidad kernel (KDE) que aproxima la campana de Gauss; el histograma semitransparente muestra la frecuencia observada. La línea roja punteada indica la media (99,88 %); las líneas grises punteadas delimitan el IC 95 % ($\pm 0,026\%$).

Cuadro 2: Límites de Capacidad (Configuración Optimizada)

Restricción	Variable	Límite Máximo
TK-100	Capacidad Geométrica	22,0m ³
TK-101	Capacidad Geométrica	35,0m ³
HX-201	Área de Intercambio	68,5m ²
EV-301	Capacidad Evaporativa	16,0m ³ /h
CF-401	Caudal Hidráulico	10,0m ³ /h
SD-501	Capacidad Evaporativa	1000kg/h

ferior a 1,5 años. Estos valores son estimaciones teóricas de diseño base; no consideran fenómenos como el ensuciamiento de membranas (fouling), la caída de presión por acumulación de sólidos o las variaciones estacionales en la composición de la soya, que requieren datos empíricos para su modelación precisa. Este resultado, junto con la tasa de éxito de 99,88 % del diseño optimizado, valida la utilidad de la arquitectura para el diagnóstico y la optimización de procesos industriales en etapa de diseño conceptual.

6. Conclusiones y Trabajo Futuro

Se presentó la arquitectura computacional de un Modelo Digital Fenomenológico diseñado para ser replicable en diferentes industrias de proceso. Las características clave que habilitan esta replicabilidad son: (i) configuración externalizada con validación automática, (ii) ecuaciones de etapa independientes y reemplazables, (iii) motor de restricciones desacoplado del modelo fenomenológico, y (iv) pruebas unitarias que verifican la

5.4 Análisis Energético

La Tabla 3 muestra el impacto de integrar ósmosis inversa (OI) como etapa de preconcentración. La OI remueve 2718,3kg/h de agua en fase líquida con solo 8,5kW eléctricos, reduciendo la carga térmica del evaporador en 836,2kW (19,5 %). El ahorro neto equivalente es de 827,7kW, lo que permite amortizar la inversión en la membrana en un periodo in-

Cuadro 3: Comparación Energética: Sin OI vs Con OI

Variable	Sin OI	Con OI
Agua removida evap. (kg/h)	6841,0	5509,0
Agua removida OI (kg/h)	0,0	2718,3
Potencia Térmica Evap. (kW)	4294,6	3458,4
Ahorro térmico (kW)	0,0	836,2 (19,5%)
Potencia Eléctrica OI (kW)	0,0	8,5
Ahorro Neto (kW)	0,0	827,7

integridad del sistema.

La revisión del estado del arte (Sección 2) reveló que, si bien existen frameworks open-source con niveles superiores de integración (Sombra y Gemelo Digital) como OpenTwins, CoFMPy, DT-Forge y UniTwin, ninguno combina simultáneamente un modelo fenomenológico basado en balances de materia y energía con validación estadística y pruebas unitarias automatizadas en el nivel de Modelo Digital. La Tabla 1 clasifica cada framework según la taxonomía de Kritzinger et al. [2], situando nuestra propuesta en su categoría real.

La instanciación de la arquitectura para una planta de ISP demostró su utilidad práctica: el diagnóstico de Monte Carlo identificó los cuellos de botella del diseño base (4,70 % de éxito), el redimensionamiento guiado por la arquitectura elevó la robustez a 99,88 % ($3,03\sigma$, ~ 1200 DPMO), y el análisis energético cuantificó un ahorro teórico del 19,5 % mediante OI.

El próximo paso obligatorio de la investigación es la validación cruzada del modelo digital con datos experimentales de planta piloto o históricos industriales. Hasta entonces, los resultados presentados constituyen una verificación numérica in silico, útil para la toma de decisiones en etapas tempranas de diseño pero no sustituta de la validación empírica. Se identifican las siguientes líneas de trabajo futuro:

- **Validación experimental:** Contraste de los balances de materia y energía con datos operativos de planta piloto para calibrar y validar el modelo fenomenológico.
- **Transición a Sombra Digital:** Instrumentación IoT y conexión con fuentes de datos en tiempo real (p. ej., mediante DT-Forge) para automatizar el flujo unidireccional de datos hacia el modelo.
- **Composición multi-planta:** Extensión del enfoque de OpenTwins para acoplar múltiples modelos digitales en una jerarquía de planta completa.
- **Modelos híbridos:** Incorporación de modelos de IA para predicción de calidad de producto, complementando los balances fenomenológicos con correcciones empíricas.

Referencias

- [1] M. Grieves and J. Vickers, "Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems," in *Transdisciplinary Perspectives on Complex Systems*, Springer, 2017, pp. 85–113.
- [2] W. Kritzinger et al., "Digital Twin in manufacturing: A categorical literature review and classification," *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018.
- [3] S. Gil et al., "Survey on open-source digital twin frameworks – A case study approach," *Software Practice and Experience*, vol. 54, no. 6, pp. 929–960, 2024.
- [4] J. Robles, C. Martín, and M. Díaz, "OpenTwins: An open-source framework for the development of next-gen compositional digital twins," *Computers in Industry*, vol. 152, p. 104007, 2023.
- [5] C. Friedrich et al., "CoFMPy: A Python Framework for Rapid Prototyping of FMI-based Digital Twins," in *Proc. 2nd Int. Conf. Engineering Digital Twins (ICEDT)*, 2025.
- [6] DT-Forge Contributors, "DT-Forge: Domain-agnostic Python framework for digital twin architectures," PyPI, 2025.
- [7] T. Häußermann et al., "UniTwin: Pushing Universal Digital Twins Into the Clouds Through Reconfigurable Container Environments," *IEEE Internet Computing*, vol. 29, no. 1, pp. 8–15, 2024.
- [8] T. Häußermann et al., "A Comparative Study of UniTwin and Established Digital Twin Frameworks for IoT Applications," *IEEE Access*, 2025.
- [9] TumorTwin Contributors, "TumorTwin: A Python framework for patient-specific digital twins in oncology," arXiv:2505.00670, 2025.
- [10] I. A. Udugama et al., "Digital twins in food processing: A conceptual approach to developing multi-layer digital models," *Digital Chemical Engineering*, vol. 6, p. 100087, 2023.
- [11] E. Gamero et al., "Data Management in Biorefineries: Conceptual Thoughts on Lean Digital Twinning," *Procedia CIRP*, 2024.
- [12] P. Verboven, T. Defraeye, A. K. Datta, and B. Nicolai, "Digital twins of food process operations: the next step for food process models?" *Current Opinion in Food Science*, vol. 35, pp. 79–87, 2020.
- [13] E. W. Lusas and M. N. Riaz, "Soy protein products: processing and use," *The Journal of Nutrition*, vol. 125, no. suppl.3, pp. 573S–580S, 1995.
- [14] ISO 23247:2021, "Digital twin framework for manufacturing," ISO, 2021.
- [15] ASME BPE-2019, "Bioprocessing Equipment Standard," ASME, 2019.
- [16] EHEDG Guidelines, "Hygienic Design Criteria," Doc. 8, 2018.
- [17] OPC Foundation, "OPC UA Companion Specification for the AAS," OPC 30270, 2022.
- [18] IEC 63278:2023, "Asset Administration Shell," IEC, 2023.
- [19] J. E. Kinsella, "Functional properties of soy proteins," *JAOCs*, vol. 56, pp. 242–258, 1979.
- [20] O. Kedem and A. Katchalsky, "Thermodynamic analysis of permeability," *BBA*, vol. 27, pp. 229–246, 1958.
- [21] Y. H. Roos, *Phase Transitions in Foods*, Academic Press, 1995.
- [22] R. H. Perry and D. W. Green, *Perry's Chemical Engineers' Handbook*, 9th ed., McGraw-Hill, 2019.